

分界点计算
 对该层级滑动窗口内的所有任务，按重叠分数从小到大排序后，找到吞吐量的临界值：
 论文中实验标定150MB/s为主机压缩效率的分界阈值，当主机压缩吞吐量低于150MB/s时，即判定为“
 主机执行效率低于CSD执行效率”；
 统计所有低于150MB/s的任务对应的重叠分数区间，取区间的上界作为该层级的重叠分数阈值：低于该
 阈值的任务会被判定为适合卸载到CSD。

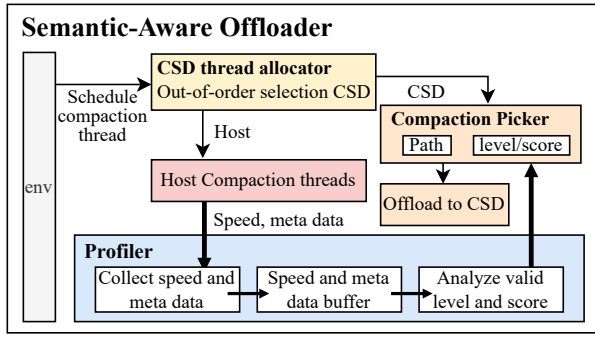


Fig. 8. Architecture of Semantic-Aware Offloader.

size=10) for each level’s compaction speed, derives level-specific overlap score thresholds.

A compaction is suitable for offloading if its level’s average speed falls below the CSD execution speed threshold or its overlap score is below the level’s overlap score threshold. This dual-threshold approach ensures efficient job selection while accounting for both computational and I/O characteristics.

To effectively distribute tasks across the CSD array, we introduce a CSD Thread Allocator that activates after compaction scheduling. The allocator employs an out-of-order selection policy to assign compaction threads to idle CSDs, ensuring workload balance. When no idle CSD is available, the system defaults to host execution.

Once a CSD is selected as the computing unit for the compaction thread, we use a Compaction Picker to select the files included in the compaction for CSD. The Profiler obtains the range of offloading levels and overlap scores boundary by collecting and calculating host compaction information. We select the files for the compaction based on these ranges and the path of the selected CSD. During the selection process, we traverse each level from **higher** to **lower** levels. If the corresponding level is not an offloading level, we select SSTables that meet the overlap score and are located on the corresponding CSD as the input files for this compaction; if the corresponding level is an offloading level, we select SSTables located on the corresponding CSD as the input files for this compaction. After the files are selected, we also check whether the selected compaction meets the resource requirements of the CSD. If it does, the compaction is offloaded to the CSD for computation; if not, it is returned to the host for computation.

In summary, our Compaction Scheduler employs intelligent job selection to optimize near-storage processing, strategically offloading compaction tasks that benefit most from CSD execution. This design achieves dual objectives: (1) maximizing computational storage resource utilization while (2) improving overall system throughput through balanced host-CSD workload distribution.

C. Compaction Acceleration Kernel in CSDs

While prior research has demonstrated FPGA-based compaction acceleration on discrete devices with abundant resources, Computational Storage Drives (CSDs) impose stringent computational and memory constraints that limit parallelism. To address this, we propose a lightweight acceleration

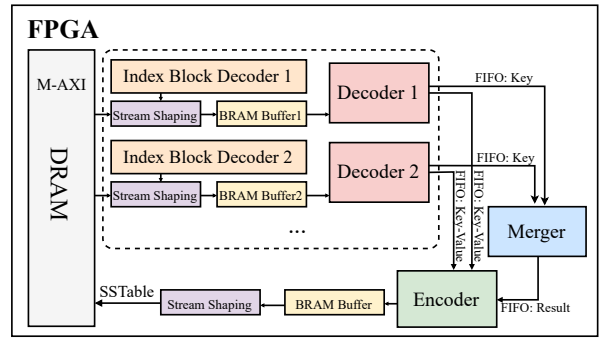


Fig. 9. Architecture of the Compaction Acceleration Kernel.

kernel specifically optimized for CSD environments. Departing from the conventional strategy of scaling performance with multiple compute units, an approach infeasible due to the limited DRAM for staging SSTables on CSDs, our design maximizes the efficiency of a single unit through aggressive intra-kernel parallelism and resource-aware optimization. Specifically, we employ widened data paths, deep pipelining, and a balanced allocation of BRAM between buffering and computation. This “scale-up” philosophy, which results in over 80% BRAM utilization to fully exploit on-chip resources, distinguishes our work and makes compaction acceleration practical under CSD constraints.

The compaction process involves decoding KV pairs from input SSTables, performing sorting and merging operations to generate updated results, and finally re-encoding the data back into the SSTable format before writing it to storage. To support this workflow, our design incorporates three primary modules: decoder, which decodes SSTables and extracts KV pairs; merger, which performs key ordering and merging; and encoder, which re-encodes KV pairs into the SSTable structure. The architecture of the acceleration kernel is illustrated in Figure 9. Similar to existing FPGA-based compaction solutions, our design follows a dataflow-driven model that exploits the inherent streaming nature of compaction. The three modules are interconnected via FIFOs, and computation proceeds at the granularity of individual KV pairs. Data availability within the FIFOs triggers the execution of subsequent stages, enabling a deeply pipelined execution pattern.

During execution, each decoder is responsible for one input SSTable. It begins by parsing the footer and index block to locate all data blocks, which are then loaded into BRAM to reduce redundant memory accesses. Once a KV pair is decoded, its key is pushed into the FIFO connected to the merger, while the full KV pair is sent to the encoder FIFO. The merger maintains a BRAM buffer for each input SSTable, compares the buffered keys, and selects the smallest key among them. It then forwards the corresponding SSTable identifier to the encoder and fetches the next key from the associated decoder FIFO. Based on the merger’s selection, the encoder retrieves the corresponding KV pair, re-encodes it into SSTable format, and updates the index and other metadata.

To address the memory constraints, we maximize BRAM utilization to support efficient data buffering and streaming.

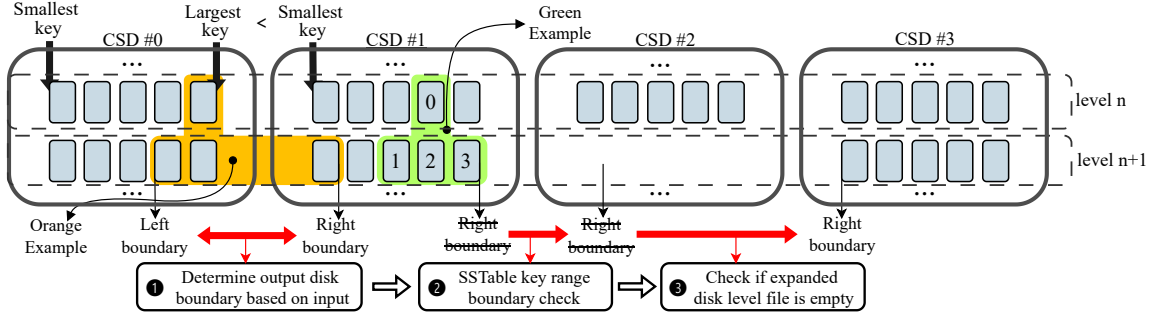


Fig. 10. CSD-Friendly Data Partitioning Scheme.

To improve DRAM bandwidth efficiency, the decoder and encoder modules utilize stream reshaping to aggregate fragmented memory requests into burst-aligned accesses matching the AXI interface width. We further enhance intra-kernel performance by widening internal data paths, deepening pipelines, and carefully overlapping computation with memory transfers. These optimizations allow a single CU to more effectively exploit the available memory bandwidth and sustain high throughput under the DRAM constraint.

When the host activates the compaction kernel, it also provides the number of KV pairs in each input SSTable. Upon completion, the kernel returns updated metadata to the host, including the number of KVs in the output, minimum and maximum key in every SSTable, etc. This enables the host side to update and maintain metadata structures accordingly.

This lightweight, pipelined kernel achieves efficient compaction execution under tight resource constraints in the CSD, enabling more relaxed constraints on the offloading conditions and contributing to higher system throughput. Furthermore, it maintains compatibility with the KV store metadata management on the host, ensuring consistency and correctness.

D. CSD-Friendly Data Partitioning Scheme

When deploying the LSM-tree based KV store on CSD arrays, we aim to achieve the following two objectives: (i) for compaction executed on the host, expose sufficient disk-level parallelism to fully utilize the aggregate I/O bandwidth of multiple CSDs; and (ii) for compaction offloaded to CSDs, minimize inter-device data transfers, as excessive cross-device traffic hampers the efficient use of each CSD’s internal data transfer resources. To meet these requirements, we propose a CSD-Friendly Data Partitioning Scheme, which consists of two main components: data placement and load balancing.

Our data placement strategy partitions SSTables within each level and distributes them across multiple CSDs. The partitioning scheme employed at each level is dictated by the corresponding data layout strategy. In LSM-based key-value stores, SSTables at each level typically adhere to one of two layout strategies: Tiering or Leveling.

For levels adopting the **Tiering** strategy, we employ a round-robin placement scheme to distribute SSTables within the level. Tiering allows overlapping key ranges among SSTables in the same level, which improves write efficiency but significantly increases the cost of a single compaction. Consequently, higher levels often use Tiering layouts. However,

TABLE I
NOTATIONS USED IN THE ILP MODEL

Parameters	Description
n	Number of CSDs
m	Number of output SSTables
a_i	Number of SSTables in the output level of the i -th CSD
d_i	Number of SSTables written to the i -th CSD
l, r	Valid range of output CSDs

as discussed in Section III, compactations at higher levels are not suitable for offloading to CSDs, and are scheduled to the host side. Therefore, for Tiering levels, our design objective is to enable greater disk-level parallelism on the host side to maximize aggregate bandwidth.

Ideally, we should evenly distribute both input and output SSTables across all devices for each compaction, thereby maximizing disk-level parallelism and fully utilizing the aggregate bandwidth. Achieving this optimal distribution, however, requires workload-aware placement that considers the current KV distribution, the load status of each device, and the input set of each compaction when deciding output destinations. While theoretically appealing, this approach introduces substantial computational overhead, especially since compactations in Tiering levels often involve a large number of SSTables.

In this work, we adopt the round-robin placement scheme, which achieves near-uniform distribution with negligible overhead. Even under imbalanced workloads, the large compaction granularity and natural redistribution effect of round-robin are sufficient to maintain balanced data placement, providing a practical and efficient solution that approaches the benefits of optimal load balancing at a fraction of the complexity.

For levels adopting the **Leveling** strategy, we determine SSTable placement based on their key ranges, ensuring ordered partitions and minimizing overlaps. In these levels, the CSD collaborates with the host to execute compaction operations. The placement policy is carefully designed to minimize inter-disk data transfers when compactations are offloaded to the CSD, thereby reducing I/O overhead and improving overall efficiency. As illustrated in Figure 10, this key range-based partitioning ensures both intra-CSD and inter-CSD SSTables in each Leveling-based level maintain a globally sorted order. Under this arrangement, compaction remains the sole operation that may change the distribution of SSTables across CSDs.